

QUIZ 2 REPORT

Sokoban Game Solver

Algorithm Design and Analysis (G)



Student:

Hisyam Syafa Raditya

(5025241130)

**INFORMATICS ENGINEERING DEPARTMENT
INSTITUT TEKNOLOGI SEPULUH NOPEMBER**

Surabaya, 15 May 2026

0 Contents

1	Introduction	2
1.1	About Sokoban	2
1.2	How This Implementation Was Built	2
1.3	Design Philosophy	2
2	Design	2
2.1	Main Menu	3
2.2	Select Level	3
2.3	Manual Gameplay	4
2.4	Victory Screen	4
3	Implementation	4
3.1	Project Structure Overview	4
3.2	Level File Format	5
3.3	The <code>SokobanSolver</code> Class	5
3.4	Dead-Space Detection	6
3.4.1	Corner Detection	6
3.5	Search Algorithm: Dijkstra's Algorithm	7
3.5.1	State Representation	7
3.5.2	The <code>solve</code> Method	7
3.5.3	Fringe Insertion: <code>_push_fringe</code>	8
3.5.4	Node Expansion: <code>_expand_node</code>	8
3.5.5	Push Validation: <code>_can_push</code>	9
3.5.6	Goal Check	9
3.6	Move Direction Constants	9
3.7	Solution Visualisation (<code>sokoban.py</code>)	9
3.8	Complexity Analysis	10
4	Evaluation	10
4.1	Strengths	10
4.2	Limitations and Potential Improvements	11
4.2.1	Scalability	11
4.2.2	Dead-Space Detection	11
4.2.3	Path Accumulation Memory	11
4.2.4	Lack of Heuristic Tie-Breaking	11
4.2.5	Game Features	11
5	Conclusion	12

1 Introduction

1.1 About Sokoban

Sokoban (Japanese for “*warehouse keeper*”) is a classic logic puzzle game first published in 1982. The player controls a character inside a warehouse and must push wooden crates onto designated goal tiles. Only one crate can be pushed at a time, and crates cannot be pulled — making every move consequential. A level is solved only when all crates are simultaneously placed on all goal positions.

Despite its simple rules, Sokoban is PSPACE-complete, meaning the difficulty scales rapidly with the number of crates and the size of the board. Even small puzzles can require dozens of carefully ordered moves, which makes the game an excellent benchmark for search algorithms in artificial intelligence.

1.2 How This Implementation Was Built

This project was built entirely in Python using two libraries:

- **Pygame** — for rendering the game window, handling keyboard and mouse input, animating sprites, and displaying UI panels such as the move counter, best-score tracker, and the controls reference.
- **heapq** (Python standard library) — for implementing the priority queue required by the search algorithm inside the solver.

The codebase is split into two files. `solver.py` is a pure-logic module with no graphical dependencies; it accepts a board description and returns an optimal sequence of moves. `sokoban.py` wraps the solver in a full Pygame application, providing a main menu, manual play mode, and an animated auto-solve visualisation.

Level data is stored in plain-text files (`level/levelN.txt`), each encoding the grid layout, starting positions, and the player’s personal best score. This format is human-readable and easy to extend with new puzzles.

1.3 Design Philosophy

The interface was designed to be clean and immediately readable. A dark gradient background keeps attention on the board, while custom pixel-art sprites differentiate walls, floors, crates, and goal tiles at a glance. Buttons on the left panel give quick access to Restart, Auto-Solve, and Exit, and a controls panel on the right eliminates the need for an external manual. The level-select screen uses a scrollable grid of cards, each showing the level number and the personal best score, so the player always knows their current progress.

2 Design

This section presents screenshots of the Sokoban game at various stages of gameplay. The screenshots below will be filled in by the author, showing each level design, the gameplay interface, and the auto-solver animation.

2.1 Main Menu

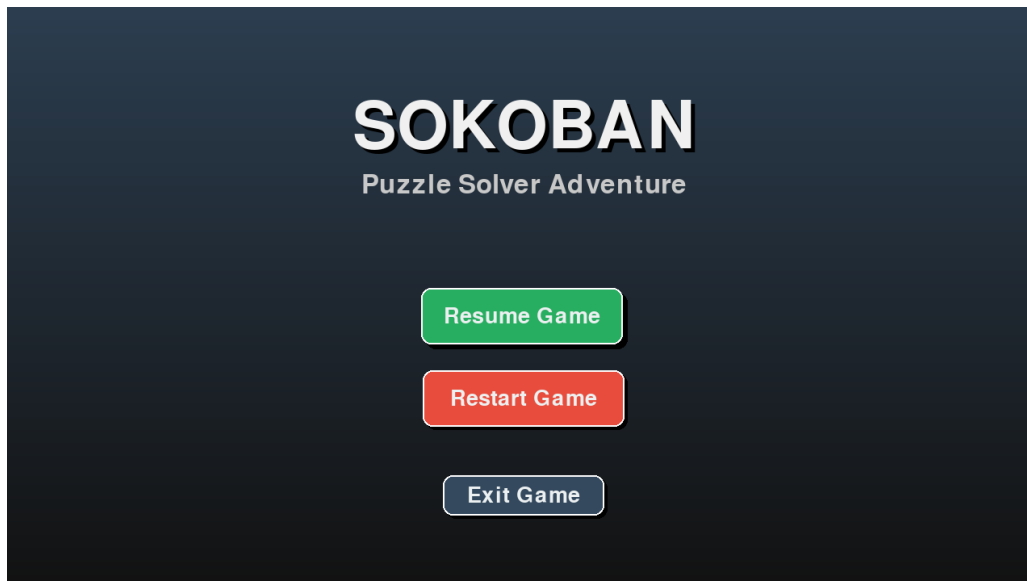


Figure 1: Main menu showing all the menu.

2.2 Select Level

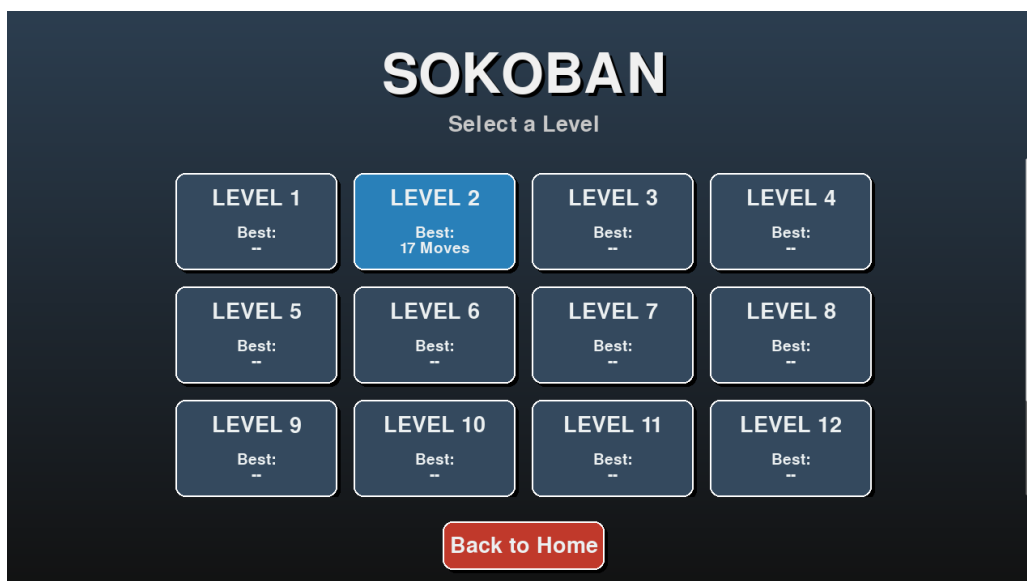


Figure 2: Select level section.

2.3 Manual Gameplay

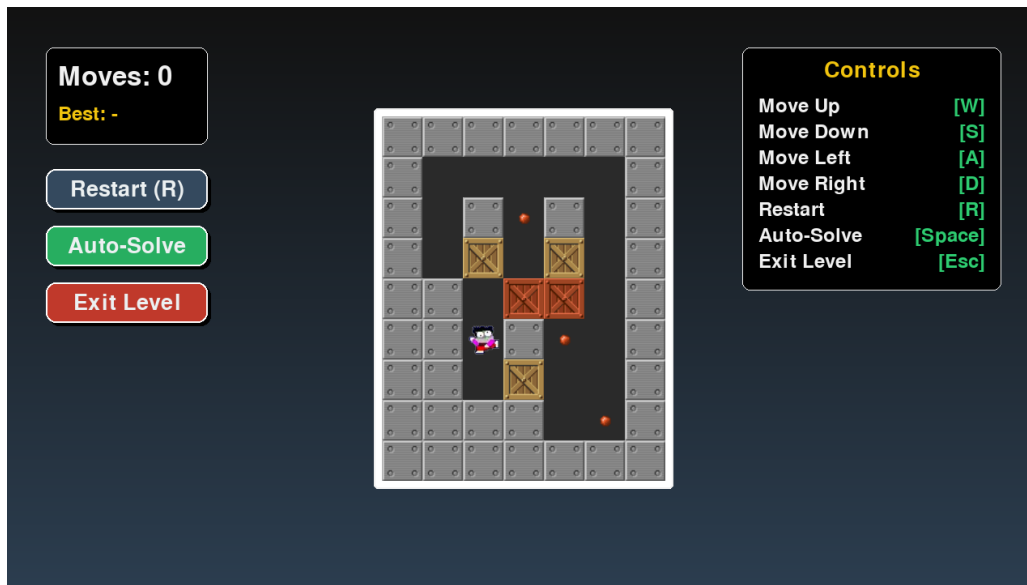


Figure 3: Player can move the character manually or use solver).

2.4 Victory Screen

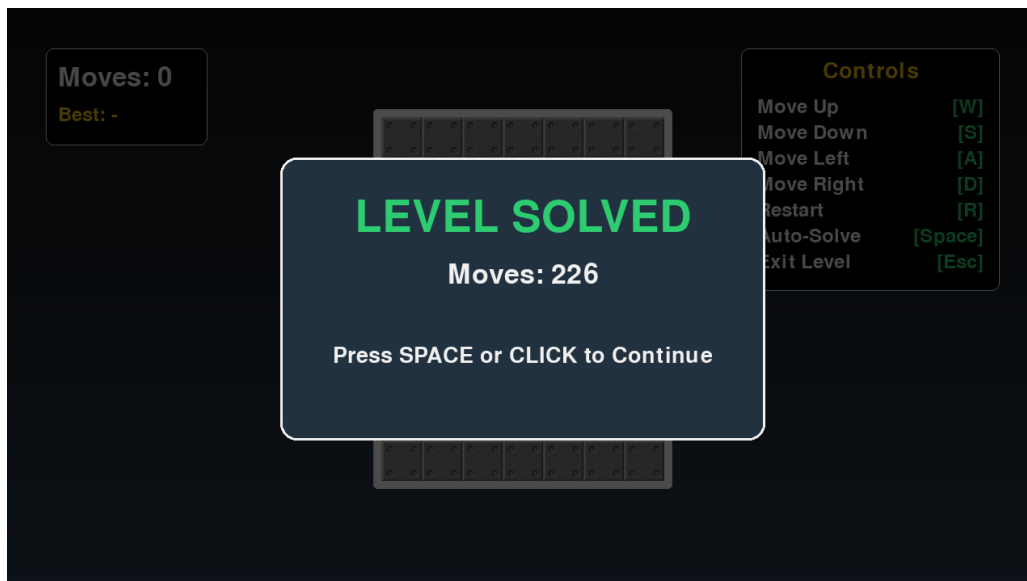


Figure 4: Victory screen displayed after all boxes are placed on goal tiles.

3 Implementation

3.1 Project Structure Overview

The project is composed of two main Python source files and a set of supporting assets:

- `solver.py` — Contains the `SokobanSolver` class, responsible for all search logic, dead-space detection, and solution computation.

- `sokoban.py` — Contains the `SokobanGame` class (Pygame-based rendering, manual play, and solution animation) along with utility functions for loading levels and managing best scores.
- `level/` — Directory with plain-text level definitions (`level1.txt` through `level16.txt`).
- `assets/` — Sprite images for walls, floor tiles, boxes, goals, and the player character.

The entry point is `sokoban.py`, which initialises Pygame, loads all levels, presents the main menu, and orchestrates transitions between manual and auto-solve modes.

3.2 Level File Format

Each level is stored as a plain-text file with the following structure:

```

1 1111111111
2 1000000001
3 1002002001
4 1000000001
5 1002002001
6 1000000001
7 1111111111
8 4,3
9 2,3;2,5;3,3;3,5
10
11 BEST_SCORE:17

```

Listing 1: Example: `level/level2.txt`

- Lines containing only 0, 1, and 2 represent the board grid: 1 = wall, 0 = empty cell, 2 = goal tile.
- The first coordinate line (4,3) is the initial player position (row, col).
- The semicolon-separated pairs (2,3;2,5;...) are the initial box positions.
- `BEST_SCORE` is an optional persisted record of the fewest moves taken in manual play.

3.3 The SokobanSolver Class

The `SokobanSolver` class (in `solver.py`) encapsulates the entire search algorithm. Its constructor receives the board grid, the initial player position, and the list of initial box positions.

```

1 class SokobanSolver:
2     def __init__(self, board, player_pos, boxes_pos):
3         self.board = board
4         self.init_player_pos = player_pos
5         self.init_boxes_pos = list(boxes_pos)
6
7         self.height = len(board)
8         self.width = len(board[0])
9
10        self.goals_pos = [
11            (r, c)
12            for r in range(self.height)
13            for c in range(self.width)
14            if board[r][c] == GOAL
15        ]
16

```

```

17         self.dead_space = self._generate_dead_space_matrix()
18
19         self.fringe      = []
20         self.visited     = set()
21         self.solution    = None
22         self.expanded_nodes_count = 0
23         self.visited_nodes_count = 0
24         self.time_used   = 0

```

Listing 2: SokobanSolver constructor

During initialisation:

1. The goal positions are extracted from the board by scanning every cell for the value `GOAL ('2')`.
2. The *dead-space matrix* is pre-computed once (see Section 3.4), avoiding repeated checks during search.
3. Statistical counters (`expanded_nodes_count`, `visited_nodes_count`, `time_used`) are initialised to zero for benchmarking purposes.

3.4 Dead-Space Detection

A box that moves into a *dead space* can never be pushed onto a goal tile; continuing to search such states wastes time and memory. The solver pre-computes a boolean matrix that marks every dead cell before the search begins.

3.4.1 Corner Detection

The primary dead-space criterion used is the *corner rule*: if a non-goal cell is surrounded by walls on two adjacent sides (forming a corner), any box pushed there is permanently stuck.

```

1  def _generate_dead_space_matrix(self):
2      ds = [[False] * self.width for _ in range(self.height)]
3
4      for r in range(self.height):
5          for c in range(self.width):
6              if self.board[r][c] == WALL:
7                  ds[r][c] = True
8                  continue
9              if self.board[r][c] == GOAL:
10                 continue # goal cells are never dead
11             if self._is_corner(r, c):
12                 ds[r][c] = True
13
14         return ds
15
16  def _is_corner(self, r, c):
17      if not self._valid_inner(r, c):
18          return False
19      return (
20          (self.board[r-1][c] == WALL and self.board[r][c-1] == WALL)
21      or
22          (self.board[r-1][c] == WALL and self.board[r][c+1] == WALL)
23      or

```

```

22         (self.board[r+1][c] == WALL and self.board[r][c-1] == WALL)
    or
23         (self.board[r+1][c] == WALL and self.board[r][c+1] == WALL)
24     )

```

Listing 3: Dead-space matrix generation

The four Boolean clauses in `_is_corner` correspond to the four possible corner orientations (top-left, top-right, bottom-left, bottom-right). A cell on the very edge of the grid (i.e., not a valid *inner* cell) is excluded by `_valid_inner`, which checks that $0 < r < \text{height} - 1$ and $0 < c < \text{width} - 1$.

3.5 Search Algorithm: Dijkstra's Algorithm

The solver uses **Dijkstra's Algorithm** (uniform-cost search) to find a *shortest-move* solution. Each state in the search is a pair (*player*, *boxes*), where *boxes* is a sorted tuple of box coordinates (making the state hashable and order-independent). The cost *g* equals the number of moves taken so far; no heuristic is used.

3.5.1 State Representation

`state = (player_pos, sorted_boxes_tuple)`

Both components must be immutable so they can be stored in the `visited` set. Box positions are always sorted before hashing, so two states with boxes in different orderings but identical positions are treated as the same state.

3.5.2 The solve Method

```

1  def solve(self):
2      t0 = time.time()
3
4      start_boxes = tuple(sorted(self.init_boxes_pos))
5      self._push_fringe(0, self.init_player_pos, start_boxes, "")
6
7      while self.fringe:
8          g, player, boxes, path = heapq.heappop(self.fringe)
9          self.visited_nodes_count += 1
10
11         state = (player, boxes)
12         if state in self.visited:
13             continue
14         self.visited.add(state)
15
16         if self._is_goal_state(boxes):
17             self.solution = path
18             self.time_used = time.time() - t0
19             return path
20
21         self._expand_node(g, player, boxes, path)
22
23     self.time_used = time.time() - t0
24     return None    # no solution exists

```

Listing 4: Main solve loop (Dijkstra)

Key observations:

- `heapq` (Python's min-heap) acts as the priority queue. Since all edges have weight 1 (one move), the heap always pops the state with the smallest g .
- A *lazy deletion* strategy is used: a state is skipped if it was already processed (`state in self.visited`), rather than updating existing heap entries.
- The path string (e.g. "RRDU") is accumulated incrementally, making trace-back trivial.

3.5.3 Fringe Insertion: `_push_fringe`

```

1 def _push_fringe(self, g, player, boxes, path):
2     node = (g, player, boxes, path)
3     self.expanded_nodes_count += 1
4     heapq.heappush(self.fringe, node)

```

Listing 5: Pushing a node onto the priority queue

The tuple $(g, player, boxes, path)$ is pushed as a single heap element. Python's `heapq` uses the first element (g) for ordering; ties are broken by subsequent tuple elements in lexicographic order.

3.5.4 Node Expansion: `_expand_node`

```

1 def _expand_node(self, g, player, boxes, path):
2     px, py = player
3     boxes_list = list(boxes)
4
5     for move, (dr, dc) in DIRECTIONS.items():
6         nr, nc = px + dr, py + dc
7
8         if not self._valid_move(nr, nc):
9             continue
10
11         new_player = (nr, nc)
12         new_boxes = list(boxes_list)
13
14         if (nr, nc) in boxes_list:                # player
15             pushes a box
16             if not self._can_push((nr, nc), dr, dc, boxes_list):
17                 continue
18             idx = new_boxes.index((nr, nc))
19             new_boxes[idx] = (nr + dr, nc + dc)
20
21         new_boxes = tuple(sorted(new_boxes))
22
23         if (new_player, new_boxes) in self.visited:    # early
24             duplicate check
25             continue
26
27         self._push_fringe(g + 1, new_player, new_boxes, path + move)

```

Listing 6: Expanding a node by generating successors

For each of the four directions (R, L, D, U), the method:

1. Computes the player's new position (nr, nc) .

2. Checks validity (in bounds, not a wall).
3. If a box occupies (nr, nc) , attempts to push it one step further; the push is rejected if the destination is a wall, another box, or a pre-computed dead space.
4. Sorts the updated box tuple and performs an early visited-check before inserting into the heap, reducing unnecessary heap entries.
5. Calls `_push_fringe` with cost $g + 1$ and the move appended to the path.

3.5.5 Push Validation: `_can_push`

```

1 def _can_push(self, box_pos, dr, dc, boxes):
2     nr, nc = box_pos[0] + dr, box_pos[1] + dc
3
4     if not self._in_bounds(nr, nc):     return False
5     if self.board[nr][nc] == WALL:     return False
6     if (nr, nc) in boxes:               return False
7     if self.dead_space[nr][nc]:         return False # pruning key!
8
9     return True

```

Listing 7: Validating a box push

The dead-space check (`self.dead_space[nr][nc]`) is the critical pruning step: it prevents the solver from exploring states in which a box is pushed into a corner from which it can never reach a goal.

3.5.6 Goal Check

```

1 def _is_goal_state(self, boxes):
2     return all(self.board[r][c] == GOAL for r, c in boxes)

```

Listing 8: Goal state detection

A state is a goal state when every box occupies a cell marked as `GOAL` ('2') in the board grid.

3.6 Move Direction Constants

```

1 DIRECTIONS = {
2     'R': (0, 1),
3     'L': (0, -1),
4     'D': (1, 0),
5     'U': (-1, 0)
6 }

```

Listing 9: Direction mapping

The solver operates in *row-major* coordinates: the first element is the row offset, and the second is the column offset. Moving Down increases the row index; moving Right increases the column index.

3.7 Solution Visualisation (`sokoban.py`)

Once `solver.solve()` returns a solution path string, the `SokobanGame.visualize_solution` method replays the moves step by step inside a Pygame window, pausing for `animation_speed` seconds between each frame:

```

1 while running and step_index < len(solution_path):
2     move = solution_path[step_index]
3     delta_row, delta_col = DIRECTIONS.get(move, (0, 0))
4
5     next_pos = (current_pos[0] + delta_row,
6                current_pos[1] + delta_col)
7     next_boxes = current_boxes[:]
8
9     if next_pos in next_boxes:                                # box is being
        pushed
10        next_box_pos = (next_pos[0] + delta_row,
11                        next_pos[1] + delta_col)
12        next_boxes[next_boxes.index(next_pos)] = next_box_pos
13
14    current_pos = next_pos
15    current_boxes = next_boxes
16
17    self._draw_board(screen, current_pos, current_boxes)
18    pygame.time.wait(int(animation_speed * 1000))
19    step_index += 1

```

Listing 10: Solution playback loop (simplified)

3.8 Complexity Analysis

Let N be the number of cells, B the number of boxes, and G the number of goal tiles.

- **State space:** The state is a pair (player cell, box positions). The theoretical maximum number of states is $O(N \cdot \binom{N}{B})$, which grows rapidly with B .
- **Time complexity:** Dijkstra with a binary heap runs in $O(|S| \log |S|)$ where $|S|$ is the number of states actually explored.
- **Space complexity:** $O(|S|)$ for the visited set and the fringe.
- **Dead-space pruning:** By preventing boxes from being pushed into corners (a common dead end), the effective state space $|S|$ is significantly reduced in practice.

4 Evaluation

4.1 Strengths

- **Correctness:** Dijkstra's algorithm is *complete* (it always finds a solution if one exists) and *optimal* (the first solution found has the minimum number of moves).
- **Clean state representation:** Using a sorted tuple for box positions and storing the full path string per node makes the code easy to read and trace.
- **Dead-space pruning:** The corner-detection pruning substantially reduces the number of states explored, especially in levels with many walls and tight corridors.
- **Separation of concerns:** The solver (`solver.py`) and the game (`sokoban.py`) are cleanly separated, making the solver reusable in headless or batch contexts.

4.2 Limitations and Potential Improvements

4.2.1 Scalability

Dijkstra's algorithm does not use a heuristic. For levels with many boxes or large grids, the search space can become prohibitively large because the algorithm expands states in all directions equally.

Improvement: Replace Dijkstra with **A* Search** and a well-designed admissible heuristic, such as the *sum of minimum Manhattan distances* from each box to its nearest goal. This can reduce the number of expanded nodes by orders of magnitude on harder levels.

4.2.2 Dead-Space Detection

The current dead-space implementation only detects simple corner positions. It misses more complex deadlocks such as:

- *Freeze deadlocks*: Two or more boxes mutually blocking each other.
- *Corridor deadlocks*: A box stuck against a wall segment with no goal in the same column or row.

Improvement: Implement *freeze deadlock detection* (checking whether a box cannot be moved in any direction) and *line deadlock detection* (checking whether a row or column between two walls contains no goal).

4.2.3 Path Accumulation Memory

Each heap node stores the complete path string accumulated so far (e.g. "RRDULD..."). On long solutions this can be a significant memory overhead, as each node duplicates the prefix.

Improvement: Store only a *parent pointer* per node and reconstruct the path after the goal is found by tracing back the chain.

4.2.4 Lack of Heuristic Tie-Breaking

When multiple states share the same cost g , Python's heap breaks ties lexicographically by comparing the subsequent tuple elements (player position, box tuple, path string). This is arbitrary and not guided towards the goal.

Improvement: Add a secondary priority key, for example the sum of Manhattan distances from boxes to their nearest goals, to guide the search more efficiently in case of ties.

4.2.5 Game Features

- **Undo move**: Manual play currently has no undo functionality. Adding an undo stack would greatly improve the playability.
- **Timer**: A visible in-game timer would add a competitive dimension alongside the move counter.
- **Level editor**: A simple GUI level editor would allow users to create and test custom puzzles without editing raw text files.

- **Solver progress feedback:** During long solving runs there is no progress indicator. Displaying the number of nodes explored in real time would improve the user experience.

5 Conclusion

This project provided valuable hands-on experience with applying graph-search algorithms to a real combinatorial puzzle. Several important lessons were learned during development:

1. **Dijkstra’s algorithm guarantees optimality:** Because each move has a uniform cost of 1, uniform-cost search always finds the solution with the fewest moves. This confirms the theoretical property that Dijkstra (and UCS) are optimal for non-negative, uniform-weight graphs.
2. **State design is critical:** Choosing the right state representation—a sorted tuple of box positions combined with the player position—is essential for correctness. Without sorting, equivalent box configurations would be treated as distinct states, causing exponential blowup.
3. **Pruning has a big impact over performance:** The dead-space pruning showed that even a simple structural insight (a box in a corner is stuck forever) can massively reduce the search effort. This reinforces the importance of *domain knowledge* in making general search algorithms practical.
4. **Heuristics matter for scalability:** The absence of a heuristic function becomes a bottleneck on larger levels. Upgrading to A* with an admissible heuristic is the natural next step to make the solver competitive on harder Sokoban puzzles.
5. **Software architecture:** Separating the solver logic from the Pygame rendering code made it straightforward to test the solver independently and to swap out algorithms without touching the game code. This principle of *separation of concerns* is a valuable design pattern for any software project.

Overall, building a Sokoban solver from scratch deepened the understanding of how algorithmic theory (graph search, priority queues, state-space exploration) translates into practical software, and highlighted the interplay between algorithm design, data structures, and domain-specific pruning.